CodeArchGuard: A GraphRAG-Powered Codebase Architecture & Refactoring Impact Assistant

Prepared By:

- Eyad Mohamed Helmy
- Mohamed El-Tanahy
- Ahmed Salah
- Mohannad Ehab
- Salma Nashaat
- Ahmed Yasser

1. Project Planning & Management

1.1 Project Idea & Executive Summary

CodeArchGuard is a GraphRAG-powered codebase architecture and refactoring impact assistant built for multi-file Python projects. In modular Python systems, modifying a core utility function or class—such as a data validation helper, cryptographic verifier, configuration handler, or shared decorator—frequently introduces unintended breaking changes across upstream workflows, scripts, and entry points multiple hops away. Additionally, critical architectural constraints (such as strict execution timeouts, memory bounds, or concurrency rules) are rarely documented inside the low-level helper being edited; instead, they reside in upstream docstrings or standalone Markdown Architecture Decision Records (ADRs).

Standard IDE reference search tools only display single-hop syntactic call lines without reading architectural documentation or selecting regression tests. Conversely, standard Vector RAG coding assistants fail to trace cross-file execution dependencies due to vocabulary mismatch between low-level utility modules and high-level business domains.

CodeArchGuard bridges this gap by combining **deterministic static code analysis** with **graph-scoped Retrieval-Augmented Generation (GraphRAG)**. Given a pinned Git repository snapshot and a selected Python function or class, CodeArchGuard:

1. Traces the verified multi-hop upstream blast radius up to top-level system entry points using a Neo4j call graph.
2. Generates a ready-to-copy `pytest` CLI command targeting all unit and integration tests linked to the impacted call path.
3. Synthesizes a grounded architectural design checklist from docstrings and ADRs strictly attached to the affected subgraph, backed by a deterministic file-and-line citation verifier.

1.2 Project Objectives

1. **Construct a Zero-Token Structural Code Graph:** Develop an offline static ingestion pipeline using Python's `ast` module and `jedi` library to extract functions, classes, module entry points, and `pytest` cases into a Neo4j property graph with exact file paths and line numbers, consuming zero LLM tokens.
2. **Compute Verified Multi-Hop Blast Radiuses:** Implement cycle-safe reverse Breadth-First Search (BFS) traversals in Cypher to identify all direct and transitive upstream callers (1 to 3 or more hops) of any modified Python symbol.
3. **Automate Targeted Regression Test Selection:** Traverse graph relationships between impacted functional nodes and test suite nodes to compile an immediate, deduplicated `pytest` execution command.
4. **Deliver Graph-Scoped Architectural Explanations:** Link Markdown ADRs and Python docstrings to graph nodes, embed them using a local text embedding model, and restrict vector retrieval strictly to documentation attached to the impacted execution path.
5. **Enforce Deterministic Citation Verification:** Implement a post-generation verification filter that cross-checks every LLM-cited file path and line number against Neo4j node properties to prevent hallucinated references.

1.3 Project Scope & Engineering Boundaries

Scope Dimension	In-Scope (Delivered by CodeArchGuard)	Out-of-Scope (Explicit Engineering Boundaries)
Target Language & Frameworks	Modular Python 3.10+ repositories (including software libraries, data/ML pipelines, CLI tools, and services) with `pytest` test suites.	Non-Python or polyglot codebases (e.g., C++, Java, TypeScript) and legacy Python 2 syntax.
Static vs. Dynamic Reachability	Deterministic static reachability: AST definitions, class methods, decorators, and cross-file imports resolved via `jedi`.	Dynamic runtime constructs (`eval()`, `exec()`, `getattr()`, dynamic string imports)—detected and flagged as explicit UI warnings rather than traced at runtime.
Documentation Sources	In-repository Python docstrings, inline module comments, and Markdown Architecture Decision Records (`.md` files).	External cloud-hosted knowledge bases requiring OAuth integration (e.g., Jira, Confluence, Notion).
Indexing Granularity	Commit-pinned Git repository snapshots to guarantee reproducible graph states and exact line-number citations.	Real-time keystroke-level background indexing inside an IDE extension.

1.4 Execution Timeline & Roadmap (7-Week Plan to Dec 1 Delivery)

Phase	Duration	Key Milestones	Concrete Deliverables
Phase 1: Foundation & Code Graph	Weeks 1–2	• Define Neo4j property graph schema and constraints. • Build Python `ast` and `jedi` parser for cross-file symbol resolution. • Implement reverse BFS caller traversal queries.	Functional offline indexer capable of ingesting a Python repository into Neo4j and returning verified multi-hop callers via Cypher.
Phase 2: Docs RAG & Initial UI	Weeks 3–4	• Extract docstrings and Markdown ADRs and attach them to code nodes. • Generate local vector embeddings for linked documentation. • Build initial web interface for repository querying.	End-to-end prototype connecting the Neo4j call graph with vector embeddings and an initial interactive UI.
Phase 3: Routing, Pytest & Verification	Weeks 5–6	• Implement the Smart Retrieval Router (Graph BFS, Vector RAG, Hybrid). • Build the targeted `pytest` CLI generator and Citation Verifier. • Validate on a controlled Python sandbox project and an open-source repo.	Integrated system featuring automated test command synthesis, hallucination filtering, and the in-app Quality Comparison Panel.
Phase 4: Polish & Final Defense	Week 7	• Run edge-case safety tests (mutual recursion, duplicate symbol names).	Production-ready graduation deliverable, complete technical documentation,

Phase	Duration	Key Milestones	Concrete Deliverables
		• Finalize system documentation and UI refinements. • Record backup demo video and rehearse graduation presentation.	benchmark results, and final defense deck.

1.5 Team Structure, Roles & Responsibilities

The team collaborates flexibly across the pipeline from setup through integration, with primary module ownership assigned as follows to maintain clear accountability:

Team Member	Designated Role	Primary Responsibilities & Deliverables
Eyad Mohamed	System Architect & GraphRAG	Overall pipeline architecture, Smart Retrieval Router logic, Hybrid GraphRAG prompt synthesis, and deterministic Citation Verifier implementation.
Mohannad Ehab	Static Analysis & AST Engine	Python `ast` syntax parsing, cross-file import resolution via `jedi`, canonical ID generation, and dynamic construct (`eval`/`getattr`) flag detection.
Salma Nashaat	Graph Database & Cypher	Neo4j 5 schema design, batch node/edge ingestion, cycle-safe multi-hop reverse

Team Member	Designated Role	Primary Responsibilities & Deliverables
		BFS queries, and `pytest` node traversal queries.
Ahmed Yasser	NLP & Vector Retrieval	Markdown ADR parsing, docstring extraction, local embedding generation (`bge-small-en-v1.5`), and graph-scoped vector filtering.
Ahmed Salah	Frontend & Visualization	Web application interface, interactive blast-radius caller tree, copy-ready `pytest` command component, and live Quality Comparison Panel.
Mohamed El-Tanahy	QA, Benchmarking & DevOps	Controlled FastAPI sandbox repository design, open-source repository testing (HTTPX/Flask), latency/recall evaluation, and containerization.

1.6 Risk Assessment & Mitigation Plan

Risk ID	Risk Description	Likelihood	Impact	Mitigation & Contingency Plan
R-01	**Cyclic Call Chains (Mutual Recursion):** Recursive functions (`A -> B -> A`) cause infinite traversal loops during reverse BFS queries.	Medium	High	Enforce path uniqueness constraints and bounded hop depth limits in Cypher queries, verified via dedicated recursion unit tests.

Risk ID	Risk Description	Likelihood	Impact	Mitigation & Contingency Plan
R-02	**Duplicate Function Name Collisions:** Identical function names across separate modules (e.g., `module_a/utils.py:verify` vs. `module_b/utils.py:verify`) merge into false edges.	High	High	Resolve all imports via `jedi` and key every Neo4j node by a canonical fully qualified identifier (`commit_hash::file_path::Class.function_name`).
R-03	**LLM Hallucinated File or Line Citations:** The generative model invents non-existent line numbers or misattributes an ADR rule during synthesis.	Medium	High	Pass all generated responses through a deterministic Citation Verifier that validates every `file:line` tag against the retrieved Neo4j node properties before UI rendering.
R-04	**Unresolved Dynamic Invocations:** Target code uses `eval()`, `getattr()`, or dynamic plugin registries that static analysis cannot trace.	Medium	Medium	Detect dynamic AST nodes during ingestion, set a `has_dynamic_constructs` flag on the node, and surface an explicit scope boundary warning in the UI.
R-05	**Hardware Constraints During Embedding:** Indexing large repositories causes high memory usage or slow embedding times on local machines.	Low	Medium	Utilize a lightweight pre-trained embedding model with batched processing and commit-hash caching so unchanged files are never re-embedded.

1.7 Key Performance Indicators (KPIs) & Success Metrics

Metric Category	Key Performance Indicator (KPI)	Target Threshold	Verification Method
Token Efficiency	LLM tokens consumed during offline structural call-graph construction	**0 Tokens**	Offline pipeline inspection (pure `ast` + `jedi` + local embedding model).
Structural Recall	Multi-hop caller and `pytest` detection accuracy on the Controlled Python Sandbox	**High Precision & Recall**	Automated evaluation against known ground-truth call chains in the sandbox repository.
Citation Faithfulness	Percentage of UI-rendered file paths and line numbers verified in Neo4j	**100% Verified (0% Hallucinated)**	Post-generation Citation Verifier audit logs across test queries.
System Latency	End-to-end execution time for structural BFS vs. full Hybrid GraphRAG	**Interactive Sub-Second (BFS) / Seconds (Hybrid)**	In-app Quality Comparison Panel latency telemetry on benchmark repositories.

2. Literature Review

2.1 Overview of Existing Approaches & Related Work

2.1.1 IDE Syntactic Reference Tools & Static Call Graphs (VS Code Pylance, PyCG)

Integrated Development Environments (IDEs) utilize Language Server Protocol (LSP) engines such as Pylance alongside static call-graph generators like PyCG (Salis et al., 2021) to resolve symbol definitions and direct callers.

- **Strengths:** Zero token cost, deterministic syntactic precision, and sub-second lookup speeds for direct references.
- **Limitations:**

    - *Single-Hop Syntactic Scope:* "Find All References" lists direct call sites one hop away without automatically computing transitive multi-hop paths up to top-level module entry points.
    - *Lack of Test & Documentation Synthesis:* IDE reference lists do not generate targeted `pytest` commands for transitive callers and cannot extract or summarize architectural constraints from Markdown ADRs.

2.1.2 Standard Vector RAG Coding Assistants (GitHub Copilot, Cursor)

Commercial coding assistants apply standard Retrieval-Augmented Generation (Lewis et al., 2020) by splitting source files into text chunks, embedding them into a vector space, and retrieving the top-k most semantically similar chunks to answer developer queries.

- **Strengths:** Effective for localized code completion, boilerplate generation, and conceptual questions where query terms match code identifiers.
- **Limitations:**
    - *The Vocabulary Trap:* Software execution paths cross module boundaries where terminology shifts completely. For example, modifying a low-level utility like `verify_hmac_signature()` in `core/security.py` directly impacts high-level workflow functions in `services/orders.py`. Because the high-level module discusses domain concepts without ever mentioning the word *"HMAC"*, cosine similarity search fails to retrieve the impacted upstream module.
    - *Unverified Citations:* Without a grounded structural graph, standard RAG assistants guess multi-hop connections and frequently hallucinate line numbers.

2.1.3 General-Purpose LLM-Extracted GraphRAG (Microsoft GraphRAG)

Microsoft GraphRAG (Edge et al., 2024) improved multi-hop reasoning over unstructured text corpora by prompting an LLM to extract entities, relationships, and community summaries during indexing.

- **Strengths:** Demonstrates strong improvements over naive Vector RAG when synthesizing information across disconnected text documents.
- **Limitations:**

- *High Token Cost & Probabilistic Edges on Code:* Relying on an LLM to read source code and guess call edges consumes substantial token budgets and introduces extraction errors, whereas programming languages possess formal grammars that static parsers (`ast`, `jedi`) can resolve deterministically at zero token cost.

2.1.4 Repository-Level Code Graph Research (RepoGraph)

Recent software engineering research such as RepoGraph (Ouyang et al., 2024) constructs line-level repository graphs to provide structural context to LLMs solving automated GitHub issues.

- **Strengths:** Validates that providing AST and import-resolved graph structures significantly increases LLM localization accuracy compared to flat text retrieval.
- **Limitations:** Academic frameworks like RepoGraph are designed primarily as autonomous bug-patching agents rather than interactive developer refactoring assistants—omitting targeted `pytest` CLI synthesis, Markdown ADR-to-code linking, and deterministic citation verification.

2.2 Comparative Analysis Matrix

Evaluation Criterion	IDE "Find References" (Pylance / LSP)	Standard Vector RAG (Copilot / Cursor)	LLM-Extracted GraphRAG (Microsoft GraphRAG)	CodeArchGuard (Proposed System)
Multi-Hop Transitive Blast Radius (3+ Hops)	No (Single-hop list)	No (Fails on vocabulary ...	Partial (Probabilistic LLM...	**Yes (Deterministic Cypher BFS)**
Zero-Token Offline Graph Indexing	Yes	Yes	No (High LLM indexing c...	**Yes (Static `ast` + `jedi` Parser)**
Grounded ADR & Docstring Constraint Retrieval	No	Partial (Unscoped sema...	Yes	**Yes (Graph-Scoped Vector RAG)**
Automated Targeted `pytest` Command Builder	No	No	No	**Yes (Transitive Test-Edge Traversal)**

Evaluation Criterion	IDE "Find References" (Pylance / LSP)	Standard Vector RAG (Copilot / Cursor)	LLM-Extracted GraphRAG (Microsoft GraphRAG)	CodeArchGuard (Proposed System)
Deterministic File & Line Citation Verifier	Yes	No (Prone to hallucination)	No	**Yes (Neo4j Property Verification)**
Explicit Dynamic Construct (`eval`) Warnings	No	No	No	**Yes (AST Static Scope Flags)**

2.3 How CodeArchGuard Advances Prior Work

CodeArchGuard combines compiler-grade static analysis with scoped neural retrieval to address the shortcomings identified in existing literature:

1. **Eliminating Indexing Token Costs:** Unlike Microsoft GraphRAG (Edge et al., 2024), CodeArchGuard uses Python's `ast` module and Jedi (Halter, 2023) to build the structural call graph deterministically with zero LLM calls.
2. **Overcoming the Vocabulary Trap:** Instead of relying on vector similarity to discover code dependencies, CodeArchGuard uses Neo4j reverse BFS to trace the exact execution path first, and restricts dense vector search strictly to ADRs and docstrings attached to that verified path.
3. **Actionable Refactoring Verification:** Extending beyond RepoGraph (Ouyang et al., 2024), CodeArchGuard automatically traverses `TESTS` edges to produce an executable `pytest` command and enforces citation accuracy via a deterministic verifier.

2.4 Documented References

- **Salis et al. (2021):** V. Salis, T. Sotiropoulos, P. Louridas, D. Spinellis, and D. Mitropoulos, "PyCG: Practical Call Graph Generation in Python," in *Proceedings of the 43rd International Conference on Software Engineering (ICSE)*, Madrid, Spain, 2021, pp. 1646–1657. (Paper Link)
- **Lewis et al. (2020):** P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474. (Paper Link)

- **Edge et al. (2024):** D. Edge et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," *arXiv preprint arXiv:2404.16130*, Microsoft Research, 2024. (Paper Link)
- **Ouyang et al. (2024):** S. Ouyang et al., "RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph," *arXiv preprint arXiv:2410.14684*, 2024. (Paper Link)
- **Xiao et al. (2023):** S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packaged Resources To Advance General Chinese Embedding (BGE Embedding Models)," *arXiv preprint arXiv:2309.07597*, Beijing Academy of Artificial Intelligence (BAAI), 2023. (Paper Link)
- **Halter (2023):** D. Halter, "Jedi: An Awesome Autocompletion, Static Analysis and Refactoring Library for Python," *Official Documentation & Repository*. (Documentation Link)

3. Requirements Gathering & Specification

3.1 Stakeholder Analysis & Needs Assessment

Stakeholder Group	Role Description	Core Pain Points & Needs	Value Delivered by CodeArchGuard
Backend Software Engineers	Engineers modifying shared functions, classes, pipelines, and utilities in multi-file Python projects.	Cannot easily see transitive upstream API callers or hidden design constraints; waste time guessing which tests to run.	Interactive multi-hop blast-radius map with exact file/line numbers, grounded warnings, and a copy-ready `pytest` command.
Software Architects & Tech Leads	Senior engineers responsible for system reliability, SLAs, and Architecture Decision Records (ADRs).	Critical constraints documented in ADRs (e.g., execution timeout limits or architectural boundaries) are overlooked during everyday code edits.	Automated linking and retrieval of ADR rules and upstream docstring warnings whenever a connected function is inspected.

Stakeholder Group	Role Description	Core Pain Points & Needs	Value Delivered by CodeArchGuard
QA & Release Engineers	Engineers verifying pull requests and maintaining CI/CD test pipelines.	Running full test suites is slow, while manual test selection misses transitive upstream breakages.	Deterministic identification of all `pytest` test functions connected to any node in the upstream blast radius.
Academic Evaluators & Reviewers	Faculty and industry examiners evaluating architectural rigor and retrieval quality.	Need empirical proof that Hybrid GraphRAG outperforms standard Vector RAG without hallucinating citations.	Built-in Quality Comparison Panel showing router decisions, query latency, verified citations, and side-by-side RAG outputs.

3.2 Functional Requirements (FR)

3.2.1 Offline Repository Indexing Stage

- **FR-01 (Git Snapshot Pinning):** The system shall ingest a local or cloned Python Git repository pinned to a specific commit hash so that all extracted file paths and line numbers remain immutable and reproducible.
- **FR-02 (Static AST & Import Resolution):** The system shall parse Python source files using the standard `ast` module and resolve cross-file symbol imports into canonical fully qualified names using `jedi` without executing repository code.
- **FR-03 (Neo4j Property Graph Construction):** The system shall store extracted entities in Neo4j using distinct node labels (`Function`, `Class`, `EntryPoint`, `Test`, `DocChunk`) and directed relationships (`CALLS`, `EXPOSES`, `TESTS`, `DOCUMENTED_BY`) storing `file_path`, `start_line`, and `end_line` attributes.
- **FR-04 (Dynamic Construct Detection):** The system shall identify dynamic execution primitives (`eval()`, `exec()`, `getattr()`) during AST walks and record a `has_dynamic_warning` attribute on the enclosing function node.
- **FR-05 (Local Documentation Linking & Embedding):** The system shall extract module/function docstrings and Markdown ADRs, create `DOCUMENTED_BY` edges linking them to relevant code nodes, and compute dense vector embeddings using a local text embedding model.

3.2.2 Online Query Routing & Hybrid Retrieval Stage

- **FR-06 (Smart Retrieval Router):** The system shall classify user requests into one of three execution pipelines:
    a. *Graph BFS Mode* for purely structural caller and entry-point listing queries.
    b. *Vector RAG Mode* for broad conceptual documentation queries.
    c. *Hybrid GraphRAG Mode* for refactoring impact analysis and architectural constraint queries.
- **FR-07 (Cycle-Safe Multi-Hop Blast-Radius Traversal):** Given a target function or class, the system shall execute a reverse BFS query in Neo4j to retrieve all upstream callers and entry points across 1 to 3 or more hops while preventing infinite loops on recursive call cycles.
- **FR-08 (Targeted `pytest` Command Generator):** The system shall traverse `TESTS` edges pointing to any node within the active blast radius and output a deduplicated, ready-to-run `pytest` CLI command specifying the exact test files and test functions.
- **FR-09 (Graph-Scoped Vector Search):** In Hybrid GraphRAG Mode, the system shall restrict vector similarity search strictly to `DocChunk` and ADR nodes attached to the retrieved blast-radius subgraph.
- **FR-10 (Deterministic Citation Verifier):** Prior to displaying the final architectural explanation, the system shall validate every cited file path and line number against the Neo4j subgraph and strip or flag any unverified citation.

3.2.3 User Interface & Evaluation Stage

- **FR-11 (Interactive Impact Explorer UI):** The web interface shall enable users to select a modified Python symbol and inspect the multi-hop caller hierarchy, clickable file paths, exact line numbers, dynamic code warnings, and the generated `pytest` command.
- **FR-12 (Grounded Design Checklist Display):** The interface shall display synthesized architectural constraints and design warnings alongside their verified ADR and docstring source citations.
- **FR-13 (In-App Quality Comparison Panel):** The interface shall provide a diagnostic view displaying the selected router mode, response latency, and a side-by-side comparison between Hybrid GraphRAG and plain Vector RAG.

3.3 Non-Functional Requirements & Quality Attributes (NFR)

- **NFR-01 (Performance & Query Latency):**
    - Pure graph reverse-BFS caller queries shall execute in sub-second interactive time for modular Python repositories.
    - Full Hybrid GraphRAG queries (including graph traversal, scoped vector retrieval, LLM synthesis, and citation verification) shall complete within interactive response thresholds (a few seconds).
- **NFR-02 (Resource & Token Cost Efficiency):**
    - Offline structural call-graph indexing shall consume **0 LLM API tokens**.
    - Documentation embedding shall operate locally on standard hardware using a pre-trained embedding model without requiring paid external embedding APIs.
- **NFR-03 (Reliability, Cycle Safety & Hallucination Control):**
    - Graph traversal queries shall guarantee termination on repositories containing direct or mutual recursion.
    - The Citation Verifier shall enforce **100% citation precision**, guaranteeing that no fabricated file path or line range is presented as verified output.
- **NFR-04 (Security & Code Privacy):**
    - Repository ingestion shall rely strictly on static parsing (`ast` and `jedi`) and shall never import or execute untrusted code from the target repository.
    - All structural graphs and documentation embeddings shall be stored locally within the host database instance.
- **NFR-05 (Usability & Maintainability):**
    - The UI shall provide one-click copy functionality for generated `pytest` commands and clear visual distinction between verified static paths and dynamic scope warnings.
    - The system codebase shall be modularized into decoupled offline indexing, graph storage, retrieval routing, and frontend layers to support collaborative 6-member team development.